VIDEO DEMONSTRATION SCRIPT: WORD REPORT + GITHUB + LIVE AWS

Student: **Yash Janardan Sabale** (PRN: 2310423T26-27) | MITAOE Pune | DevOps IA Activity 1

Section 1: Title Page & Student Profile (0:00 – 0:45)

■■ WHAT TO SHOW ON SCREEN: Open Word Report (DevOps_IA1_CaseStudy_Report_Yash_Sabale.docx) on Page 1 showing student profile.

■■ *SPOKEN SCRIPT:*

"Hello respected evaluator. My name is Yash Janardan Sabale, PRN 2310423T26-27, Third Year B.Tech Computer Engineering at MIT Academy of Engineering, Pune. Today, I am demonstrating my submission for Internal Assessment Activity No. 1: DevOps Case Study and Ansible Automation. For this assignment, I have analyzed and implemented DevOps automation for ShalaConnect, a real-world educational governance platform for Cluster Resource Centre Heads. The backend is an enterprise service built on Java 21 LTS, Spring Boot 3.2.3, and PostgreSQL 15, deployed onto a multi-node AWS EC2 cloud architecture using Ansible."

Section 2: Rubric Compliance & Architecture Diagrams (0:45 – 2:00)

■■ WHAT TO SHOW ON SCREEN: Scroll to Rubric Table, then scroll to Figure 1 (Lifecycle), Figure 2 (Pipeline), and Figure 3 (Ansible Push Architecture).

■■ *SPOKEN SCRIPT:*

"As outlined in our rubric compliance matrix, this project fulfills all 4 evaluation criteria: complete DevOps architecture, multi-tier Ansible inventory, modular Ansible roles, and verified live execution. Here in Figure 1 is our end-to-end DevOps Lifecycle. We replaced the legacy manual deployment process with an automated 8-stage feedback loop: Plan, Code in Java 21, Build with Maven, Test with JUnit, Release to GitHub, Deploy agentlessly via Ansible, Operate under Linux systemd, and Monitor via /api/health telemetry. Figure 2 illustrates our cloud delivery pipeline: code commits trigger automated build and packaging, after which Ansible orchestrates provisioning across AWS EC2. Figure 3 details our Ansible Agentless Push Architecture. From our dedicated Control Node at Public IP 13.201.133.251, Ansible connects over OpenSSH port 22 to configure our Managed Application Node at Public IP 13.126.181.31, requiring zero agents on the target machine."

Section 3: Authentic Live Execution Evidence in Report (2:00 – 3:30)

■■ WHAT TO SHOW ON SCREEN: Scroll through Section 3 showing all 12 screenshots: AWS Console, SSH, inventory, ping, playbook recap, systemd, DB, and curl.

■■ *SPOKEN SCRIPT:*

"Now, let us review the 12 authentic execution screenshots captured during our live cloud deployment: Evidence 1 & 2 show the AWS Management Console launching our dedicated Control Node with Public IP 13.201.133.251. Evidence 3 & 4 show the target APP instance running with Public IP 13.126.181.31. Evidence 5 & 6 show SSH access from our Windows terminal into the Control Node, syncing the repo, and verifying inventory_aws_ec2.ini. Evidence 7 demonstrates the Ansible Ad-Hoc Ping: ansible all -m ping confirms successful agentless communication with ok: [ec2-app-server]. Evidence 8 & 9 capture our master playbook execution. Notice our initialization banner displaying my name: Yash Janardan Sabale (PRN: 2310423T26-27). The final PLAY RECAP confirms 37 tasks OK and zero failures (failed=0). Evidence 10 verifies the remote systemd daemon: shalaconnect.service is active and running with Tomcat on port 8080. Evidence 11 verifies PostgreSQL 15, listing all 14 JPA domain tables generated in our school_management database. Evidence 12 shows our live curl probe to http://13.126.181.31/api/health, returning HTTP 200 OK and status UP via Nginx reverse proxy."

Section 4: Codebase Walkthrough on GitHub (3:30 – 5:00)

■■ WHAT TO SHOW ON SCREEN: Switch to GitHub browser tab (YashSabale01/ShalaConnect). Open ansible.cfg, inventory_aws_ec2.ini, playbooks/site.yml, and roles/springboot_app/tasks/.

■■ *SPOKEN SCRIPT:*

"Now let us examine the version-controlled codebase hosted on my GitHub repository under DevOps_Ansible_Assignment. First, in ansible.cfg, we define global orchestration parameters: remote_user = ec2-user, privilege escalation with become = True and become_method = sudo, and optimized SSH multiplexing. In inventory_aws_ec2.ini, we define our managed target ec2-app-server at 13.126.181.31, configure SSH keys, and group webservers and app servers along with centralized port variables. Here is our master orchestration entry point, site.yml. It targets hosts: all, gathers system facts, verifies the OS family, and invokes our enterprise role. Our role is structured into modular task files following industry best practices: In setup_environment.yml, we install OpenJDK 21 Corretto, Maven, Nginx, and PostgreSQL 15, initialize the database, configure trust authentication, and create a restricted system user appuser. In deploy_code.yml, we create /opt/shalaconnect-app, deploy our executable JAR, and render production properties from Jinja2 templates. In configure_service.yml, we template our systemd unit file with automatic 5-second restart policies and process sandboxing. In configure_nginx.yml, we deploy an Nginx reverse proxy configuration forwarding external Port 80 traffic to internal Port 8080. In verify_health.yml, we automated post-deployment smoke testing using the URI module to poll the /api/health endpoint."

Section 5: Live Browser Telemetry & Conclusion (5:00 – 5:30)

■■ WHAT TO SHOW ON SCREEN: Switch to live browser tab (http://13.126.181.31/api/health) and hit Refresh.

■■ *SPOKEN SCRIPT:*

"Finally, here is the live, public verification on our running AWS EC2 instance at http://13.126.181.31/api/health. As you can see on screen, the Nginx reverse proxy forwards to Spring Boot, which queries our PostgreSQL database and returns HTTP 200 OK with JSON payload: status UP, service ShalaConnect API. In conclusion, this project successfully automated the entire infrastructure and deployment lifecycle of ShalaConnect. Deployment duration was cut from 45 minutes to under two minutes, configuration drift was eliminated, and security was hardened. All report documents, diagrams, playbooks, and evidence have been compiled and submitted. Thank you very much!"